Image Compression and Image Steganography

MATIN MONSHIZADEH[1,*] AND PARDIS BASIRI[1]

[1] The Department of Computer Science Engineering and Information Technology, Shiraz University, Iran
[*] Corresponding author: matinmonshizadeh@gmail.com

Compiled January 23, 2023

Abstract
Image compression and steganography are two topics we checked in this research. Many algorithms and techniques exist to implement them, but we have used a unique algorithm for each subject and fully described their implementation with python programming language.

INTRODUCTION

Image compression and steganography are each used in different fields and have been effective in other parts. Algorithms play a significant role in these two fields, and each should be optimally effective in this situation so that we get the desired result from them.

1. IMAGE COMPRESSION

Image compression is an application of information compression on digital images; in other words, this work aims to reduce the redundancy of photo content for the ability to store or transfer information in an optimal form [1]. Many algorithms and techniques exist for image compression. We used the Huffman coding technique to implement this part.

Huffman Coding

The history of the Huffman coding algorithm came about as the result of a class project at MIT by David A. Huffman as a student in 1951. [2]. This algorithm is lossless compression, which means this method keeps the same information as the original image. Lossless compression is also reversible and has the highest possible quality standards. This method is against lossy compression, also known as irreversible compression. Table 1 shows the comparison between lossy and lossless compression.

Table 1. Comparison Between Lossy and Lossless Compression

Lossy Compression	Lossless Compression
Also known as irreversible compression	Also known as reversible compression
Data reduction is higher	Data reduction is lower
Resultant file is smaller than the orginal	Resultant file is not as small

Huffman is widely used in all the mainstream compression formats, from GZIP, PKZIP (WinZip, etc.), and BZIP2 to image formats such as JPEG and PNG.

Method

This section describes the implementation of Image Compression Using Huffman Coding Techniques. The first step is to open and read the image as binary and store it in the string, like in figure 1.

Fig. 1. Open and read image as binary.

The second step is to separate the string in which we have stored the binary code into 8 bits chunks and get the frequency of each byte and store it in dictionary, like in figure 2.

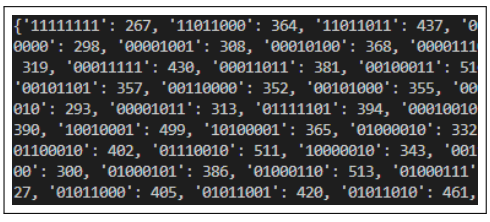

Fig. 2. Storing each byte with its frequency.

After that, we import the Heapq library to use min-heap and make a Huffman tree. Figure 3 shows each byte with its Huffman code stored in the dictionary.

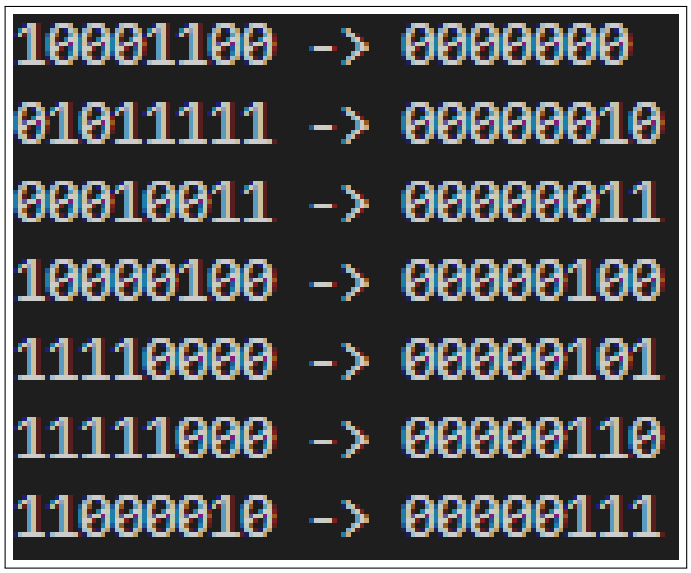

Fig. 3. Storing each byte with its frequency.

In the following, we replace the Huffman code on the first string to store compression data.
If the first string that we stored binary code of the image is S_1 and the second string that replaces the Huffman code is S_2, the compression ratio is calculated as follows:

$$CR = \frac{S_1}{S_2} \tag{1}$$

If we implement the Huffman coding algorithm correctly, the CR will be more than one.
For the last part, we implement the previous parts again in reverse to decompress the image and show that the Huffman coding algorithm is lossless compression.

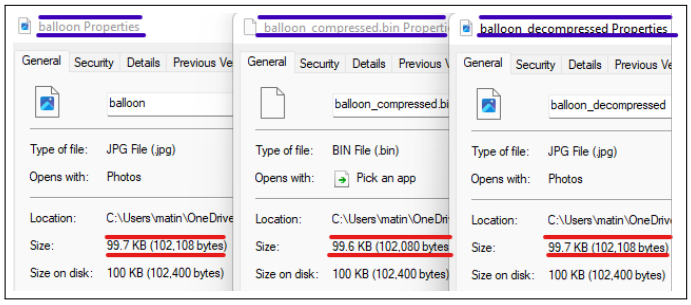

Fig. 4. Compare the size of 3 files. **Left picture** is the original image, **Middle picture** is the compressed file, and **Right picture** is the decompressed file.

Also, we show the Huffman coding algorithm in algorithm 1.

2. IMAGE STEGANOGRAPHY

Steganography is the art and science of putting secret messages in a cover message so that no one but the sender and receiver will suspect the secret message's existence. Steganography means

Algorithm 1. Huffman Coding algorithm

```
procedure HUFFMAN(C)
    n ← |C|
    Q ← C
    for i ← 1 to n − 1 do
        allocate a new node z
        left[z] ← x ← EXTRACT − MIN(Q)
        right[z] ← y ← EXTRACT − MIN(Q)
        f[z] ← f[x] + f[y]
        INSERT(Q, z)
    return EXTRACT − MIN(Q)
```

hiding a secret message behind a normal message, derived from the Greek words Steganos, meaning cover, and Graphia, meaning writing [3]. We use modern steganography techniques and tools to ensure our messages remain hidden. Also, people think that steganography is similar to cryptography. No, they are two completely different concepts, and we will point out their differences in table 2.

Table 2. Comparison Between steganography and cryptography

steganography	cryptography
Steganography means covered writing	Cryptography means secret writing
Once it has been discovered anyone can get the secret data	Once it has been discovered no one can easily get the secret data

Depending on the nature of the cover object (the actual object in which hidden data is embedded), steganography can be used in various fields, including text steganography, photo steganography, video steganography, and network steganography. There are existing algorithms for steganography; we have reviewed the LSB algorithm here.

LSB Steganography

LSB means least significant bit and it is an image steganography technique in which messages are hidden inside an image by replacing each pixel's least significant bit with the bits of the message to be hidden [4].

Method

This section describes the implementation of Image Steganography Using the LSB algorithm. At first, we have four inputs that we show in figure 5.

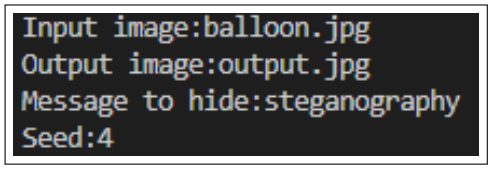

Fig. 5. Steganography inputs. An **input image** is the name of our image as input, an **Output image** is the name of the output, the **Message to hide** is the Message that we want to hide in the picture, and the **Seed** is the key that we chose.

Then, we convert our message to binary and store it. In the next step, we get the pixel coordinates and values, convert the pixel value to binary, modify the pixel value, convert the pixel value to an integer, set the pixel value, and finally save the output image and decode it and extract the message from the image.

REFERENCES

1. A. Subramanya, "Image compression technique," IEEE potentials **20**, 19–23 (2001).
2. A. Moffat, "Huffman coding," ACM Comput. Surv. (CSUR) **52**, 1–35 (2019).
3. N. Hamid, A. Yahya, R. B. Ahmad, and O. M. Al-Qershi, "Image steganography techniques: an overview," Int. J. Comput. Sci. Secur. (IJCSS) **6**, 168–187 (2012).
4. N. Jiang, N. Zhao, and L. Wang, "Lsb based quantum image steganography algorithm," Int. J. Theor. Phys. **55**, 107–123 (2016).